

Agentic SDD Harness Handbook

This handbook describes the repository-native Agentic SDD workflow and the current verification-v2 implementation in Showcase Application. The accepted feature specification, plan, schemas, and tests remain the detailed contracts; this guide is the operator entry point.

NOTE: M5.3 history: M5.3 is master-frozen after 16 Design Authority runs. The canonical M5.3 Design Gate did not pass. Explicit master closure decisions authorized implementation of the remaining plan; this handbook does not represent those decisions as a Design Gate pass.

1. Lifecycle at a glance

For small, clear work, begin with a feature specification and task DAG. For uncertain work, use Wayfinder to resolve decisions before writing the spec. A large feature can follow:

```
destination -> decision map -> reconcile -> spec/plan -> design authority
    -> independent verification contract -> task DAG -> build/evaluate
    -> integration -> human-ready closure
```

The lifecycle is evidence-based. A task is complete only after its accepted criteria have evidence and the harness records the transition. A generated packet or a successful provider response alone is not completion authority.

Useful entry points:

```
python3 tooling/agent-harness/wayfinder.py --help
python3 tooling/agent-harness/design.py --help
python3 tooling/agent-harness/verification_contract.py --help
python3 tooling/agent-harness/harness.py --help
python3 tooling/agent-harness/orchestrate.py --help
```

2. Trust and authority

The harness is local and repository-native. Accepted specifications, plans, contracts, and trusted control configuration define policy. Source code and tests are project inputs. Provider output, tracker text, and runtime output are evidence, not instructions. Secrets must not enter model context, persistent candidate identity, or ordinary logs.

The verification-v2 model has one lifecycle authority: `.agent-state`. It records accepted lifecycle transitions using compare-and-swap semantics. Immutable records under the verification-v2 control store support those transitions but cannot authorize themselves merely because a record exists. The store is resolved from the primary Git worktree at:

```
<primary-worktree>/ .agent-runs/control/verification-v2/
```

The current code contains immutable-record publication, execution journals, projections, and the supervisor/executor primitives. Integration work must keep `.agent-state` as the transition authority and treat the store as subordinate. Never infer authority from a worktree-local store, current HEAD, a cache entry, or a caller-provided identity.

3. Source authority and task execution

Verification authority starts from ordered trusted source inputs and a lifecycle-accepted Source Composition Authority. Source Resolution binds to that accepted source; it cannot select a different commit. A task packet or arbitrary Git HEAD is not source authority.

The execution sequence is Model B:

```
pre-builder authority -> builder -> privacy preflight -> candidate seal  
-> final changed surface -> applicable obligations -> execution units  
-> immutable final plan -> lifecycle acceptance -> admission -> launch
```

Historical T-003 attempt 1 predates the final Verification Execution Plan model. Preserve it as historical evidence. A future replan or recovery must establish current bindings rather than inventing missing plan, source, surface, origin, admission, or launch records for that old attempt.

4. Candidate sealing and privacy

Candidate sealing uses a versioned deterministic serialization of candidate-relevant repository state relative to its authoritative base. The surface accounts for committed delta, staged and unstaged changes, deletions, and untracked candidate files. Only the fixed trusted harness runtime/control namespace and Git metadata are excluded; .gitignore does not create an authority exclusion.

Before durable candidate hashes are published, trusted privacy checks inspect candidate state. A secret-bearing or unscannable candidate fails closed. Secret bytes, secret excerpts, and secret-derived hashes must not be persisted. Product artifacts must exist before sealing. Verification-only output belongs under the trusted runtime namespace, outside the sealed surface. A post-seal candidate mutation invalidates the old binding and requires the appropriate reseal/replan flow.

5. Declarative verification profiles

Profiles are JSON version 1 objects with a gates array. The Showcase profile is:

```
tooling/agent-harness/verification-profiles/showcase.json
```

The profile defines commands, input and applicability patterns, dependencies, sandbox and retry policy, criticality, and produced/consumed artifacts. Trusted control resolves profile identity from its trusted configuration root and validates the same opened bytes it hashes. Task packets, CLI arguments, providers, and executors cannot add trusted gates or upgrade execution origin.

Applicability is derived from the final sealed surface. Required profile criteria remain explicit obligations even if a task packet omits a similar command. Command equality does not merge obligation identity. A physical execution may satisfy multiple obligations only when they were grouped into the same canonical execution unit before launch.

Inspect and validate the Showcase profile with the feature's harness checks and tests. Do not treat its presence as proof that every gate has run.

6. Planning, fingerprints, and generated artifacts

The shared planner has three decisions:

- * `ALREADY_GREEN`: a complete PASS receipt and all required artifacts match the current fingerprint; action is REUSE.
- * `INVALIDATED_BY_THIS_PATCH`: older PASS evidence exists, but an input, policy, dependency, or artifact differs; action is always RUN.
- * `RUN_NOW`: no reusable PASS exists or policy requires fresh execution; action is RUN.

`INVALIDATED_BY_THIS_PATCH` is an explanation, never a skip. Required gates are topologically ordered with deterministic ties. The first required gate failure marks later required gates blocked-by-failure; they are not reported GREEN.

A reusable fingerprint binds the exact profile content, gate and command identity, working directory, origin policy, sandbox/retry policy, declared matched input manifest, dependency evidence, safe probes, policy checkpoint, repository identity, and final candidate/surface where applicable. Matched additions, edits, deletions, and declared generated artifacts can invalidate reuse. Symlink-affected inputs and opaque external state cannot establish reusable identity. A producer/consumer artifact is verified by its content identity; missing, overwritten, or mismatched artifacts block reuse.

Product artifacts are created before candidate sealing and become candidate state.

Verification-only output is written under the trusted runtime namespace. It cannot be written into the sealed candidate and then ignored. Raw stdout/stderr are diagnostic only and are not part of a successful evidence hash; removing logs does not invalidate a complete structured receipt. Derived cache corruption is not authority and must result in safe fresh execution or a fail-closed outcome.

7. Advisory planning and authoritative execution

The current `verify.py` plan command is advisory. It constructs a deterministic plan view and performs no command execution or lifecycle mutation:

```
python3 tooling/agent-harness/verify.py plan \
  --repo . \
  --profile tooling/agent-harness/verification-profiles/showcase.json \
  --base-sha <trusted-base-sha> \
  --family-id <family-id> \
  --policy-checkpoint <trusted-checkpoint>
```

An advisory plan is not execution authority. The authoritative direct integration interface is designed to consume an already lifecycle-accepted plan using `verify.py run --mode integration --plan-id <plan_id>`. It must not synthesize a plan from `--base`, current HEAD, task text, or an advisory result. In the current implementation snapshot, the direct run adapter fails closed with verification-blocked because the accepted-plan resolution/launch integration is not yet wired; it does not launch a verifier. Do not document or rely on this CLI path as operational until that adapter is implemented and tested.

8. Launch, ownership, and recovery

The supervisor is the process-creation boundary. Commands are parsed into argv without a shell fallback, checked against the harness allowlist, and prepared through the qualified sandbox backend. The execution store records admission, start/drainage, terminal receipt, and failure data. The executor rechecks candidate and gate readiness around execution and binds evidence to the plan/family/profile/policy and candidate surface.

The intended one-winner protocol is:

```
pre-launch admission -> launch reservation -> ownership -> durable launch
consumption -> ephemeral winning capability -> physical process
-> started/drain records -> evidence -> completion
```

An uncertain owner or unresolved journal blocks competing execution. Do not steal a lock based on age or PID guesses, reconstruct a consumed one-shot capability after a crash, or label an ambiguous launch as an ordinary verifier failure. Resolve through the existing lifecycle and recovery state. Recovery of the same exact identity may continue; replan to a new identity cannot inherit old completion evidence.

Durable lifecycle acknowledgment must follow complete atomic publication and required fsync steps. If publication or recovery is ambiguous, fail closed.

9. Evidence, coalescing, and completion

Evidence is immutable and bound to the exact execution identity: plan, family, candidate, final surface, profile/policy checkpoint, obligation and execution unit, admission, origin, launch consumption, and lifecycle generation where applicable. Cross-identity evidence reuse is forbidden. Same-identity recovery may reuse evidence only when every required binding still matches.

Distinct task-command occurrences and profile criteria remain distinct obligations. A single planned execution can satisfy pre-bound members of its execution unit; this is coalescing, not retroactive reuse. A task-origin result cannot later be relabeled independent.

Completion checks the current accepted authority and all required evidence bindings; a successful historical command alone is not sufficient.

10. Human retry authorization

A critical failure fence remains active while its canonical fingerprint remains applicable, including across replan or family changes. Replanning does not erase the fence. An old grant cannot cross plan/family/generation identity. A trusted human can issue a new exact-scope grant that identifies both the current execution context and the historical fence.

The grant envelope is signed with Ed25519. Trusted control verifies it against the allowlisted public-key issuer registry, validates scope and non-empty justification, then .agent-state consumes it once through its lifecycle transition. The private signing key is outside automation. Agents and providers may request or present grants, but cannot mint or authenticate one. --by, usernames, environment variables, provider claims, and unsigned JSON are attribution only, not proof of a human decision.

Machine outcomes remain distinct:

needs-human - A trusted human decision/action is required. (CLI exit: 4)

verification-blocked - A required authority or precondition is missing or invalid. (CLI exit:

5)

verification-owned - An exact active execution is already authoritatively owned. (CLI exit: 6)

These are control dispositions, not test failures. A grant authorizes one retry opportunity; it is neither verification evidence nor independent-origin qualification.

11. Manual evidence and telemetry

Manual evidence records observations and provenance; it cannot transition workflow state, authorize a retry, or establish independent origin by itself. Task and attempt identifiers resolve through trusted lifecycle history. Attempt-scoped evidence requires an exact attempt and a checkpoint belonging to that attempt. Never infer "latest attempt."

The current record-manual command stores a hash-bound telemetry observation only. It does not authenticate the reviewer, verify an Ed25519 manual-review attestation, satisfy a Verification Execution Plan obligation, or transition task/workflow state. In particular, supplying --plan-id currently returns verification-blocked with reason MANUAL_EVIDENCE_PLAN_BINDING_UNAVAILABLE and exit 5. Do not report a plan obligation as covered from this record.

--task-attempt is a positive numeric lifecycle attempt ordinal, not a caller-created UUID. The command resolves it against the feature's .agent-state attempt-binding history (attempt_bindings) and requires exactly one matching task; when --task is also given, it must match that resolved task. It does not choose a latest attempt. A task-only record is accepted only when the report has the matching canonical Task line and the task/role resolve against the task DAG.

After a manually run reviewer/evaluator result is used as accepted evidence, show this instruction:

MANUAL TELEMETRY REQUIRED

Then print the exact repository-native invocation for that report and its exact task/attempt/checkpoint. The report must already exist beneath the runtime manual-reports/ directory and contain the required top-level Feature, Reviewed checkpoint, Verdict, and Completed at markers; task-linked reports also require Task. The checkpoint must equal the clean primary-worktree HEAD. For example, after substituting the real values and creating the report:

```
python3 tooling/agent-harness/telemetry.py record-manual \
  --repo . \
  --feature SDD-OBS-001 \
  --role reviewer \
  --provider codex \
  --checkpoint <full-clean-head-sha> \
  --verdict PASS \
  --report .agent-runs/control/verification-v2/manual-reports/<report-file.md> \
  --task T-900 \
  --task-attempt <numeric-attempt-ordinal>
```

Omit --task-attempt only for a genuinely task-scoped observation. Do not pass --plan-id when seeking plan-obligation coverage: the current command deliberately rejects plan binding as unavailable. A recorded observation remains telemetry-only and is not authenticated

manual-review evidence. Check current flags with:

```
python3 tooling/agent-harness/telemetry.py --help
python3 tooling/agent-harness/telemetry.py record-manual --help
```

Telemetry records useful run/provider usage and duration data. Unknown provider cost stays unknown; it is not estimated from tokens. Keep logs and reports in the designated runtime namespace and never copy secrets into telemetry. Retention must preserve authority records and active references; only disposable runtime artifacts may be pruned under a deterministic policy.

12. Failure, resume, quarantine, and rollback

The established task harness treats needs-human as an explicit pause. Inspect task state and evidence before resuming:

```
python3 tooling/agent-harness/harness.py status docs/specs/<FEATURE>
python3 tooling/agent-harness/harness.py doctor
python3 tooling/agent-harness/orchestrate.py docs/specs/<FEATURE> --plan
```

Use harness.py human-resolve only for a decision within the accepted task contract. If the contract changes, revise/replan rather than smuggling the change into runtime feedback. Existing human-resolve and authorize-retry commands record task-harness decisions; their --by value is an audit label and is not cryptographic authentication for a verification-v2 critical-gate grant.

For verification-owned or uncertain process state, do not edit journals, delete control records, remove worktrees, or release ownership as a shortcut. Follow the implemented recovery command and preserve the audit history. Roll back only a disposable implementation/worktree change using the documented Git workflow; never roll back .agent-state or verification authority records by hand. A quarantine is an operational containment decision: keep the affected worktree untouched until liveness/drainage is proved and trusted lifecycle state permits recovery.

13. Compatibility, rollback, and no-remote authority

Legacy v1 provenance remains readable for aggregate run/provider/status/duration/token and known-cost summaries. A legacy record is never verification-v2 reuse authority. Unknown usage or cost stays unknown; token counts do not produce an invented cost estimate. New verification records are local and secret-minimized.

No runner, planner, evaluator, or CI path receives push, PR, deploy, merge, or remote tracker mutation authority. CI validates committed repository artifacts without invoking model providers or mutating remote state. Local source-composition preparation is distinct from committing task/product changes or updating a user branch.

If a protocol/schema change requires rollback, stop new verification, preserve unresolved journals and authority references, prove all owned processes and descendants drained, then select a compatible reporting/runtime version. An older harness may read supported legacy aggregates but cannot launch, accept plans, resume, or complete verification-v2 lifecycle state. If drainage or lifecycle state is uncertain, keep the worktree/runtime quarantined and require trusted recovery; never delete authority records to force rollback.

Retention may remove disposable logs and derived summaries/indexes only when no active or authoritative lifecycle reference depends on them. It must retain unresolved execution records, terminal critical failures, grants/consumptions, and exact evidence required to reconstruct decisions. This snapshot does not expose a general verification-v2 pruning command; do not manually delete control-store records.

14. Operating and validation commands

Common local checks:

```
python3 tooling/agent-harness/harness.py doctor
python3 tooling/agent-harness/harness.py validate docs/specs/SDD-OBS-001
python3 tooling/agent-harness/harness.py validate-all docs/specs
python3 -m unittest discover -s tooling/agent-harness/tests -p 'test_*.py' -v
```

Use focused test modules while changing a subsystem, then run the broader harness suite at integration. The repository's Java and frontend checks remain separate product gates; consult the accepted Showcase profile and current CI workflow for which commands apply to the changed surface.

Agentic SDD CI validates protocol artifacts, harness behavior, examples, and checked-in evaluations. CI does not invoke a model provider or mutate remote systems. Human/provider actions stay local and are recorded as evidence where required.

15. M5.3 implementation history and current status

The SDD-OBS-001 M5.3 candidate underwent 16 canonical Design Authority runs. The final canonical Design Gate did not pass. Master then froze M5.3 and supplied explicit closure decisions for implementation: canonical candidate serialization, durable lifecycle CAS acknowledgment, rework counting, signed human retry grants, manual attempt scope, and privacy-first candidate sealing. These decisions authorize implementation; they do not retroactively change the grill or gate history.

At the time of writing, verification-v2 implementation and integration are in progress. The modules described above include concrete profile, planner, candidate-sealing, store, supervisor, executor, grant-verification, and outcome-classification code. Features not yet wired into lifecycle acceptance or the direct CLI must remain documented as unavailable/fail-closed until implementation and tests establish otherwise.

16. Source map

- * Workflow and operator guide: this handbook.
- * Concise entry point: README.md (README.md).
- * Frozen M5.3 contract and acceptance criteria: ../specs/SDD-OBS-001/spec.md (../specs/SDD-OBS-001/spec.md) and ../specs/SDD-OBS-001/plan.md (../specs/SDD-OBS-001/plan.md).
- * Task DAG: ../specs/SDD-OBS-001/tasks.json (../specs/SDD-OBS-001/tasks.json).
- * Verification-v2 implementation: ../tooling/agent-harness/verification/ (../tooling/agent-harness/verification/).
- * Showcase gate profile:

```
../../tooling/agent-harness/verification-profiles/showcase.json  
(../../tooling/agent-harness/verification-profiles/showcase.json).  
* Agentic SDD CI: ../../github/workflows/agentic-sdd.yml  
(../../github/workflows/agentic-sdd.yml).
```